Empório da Música — Customer Service AI Agent

Technical Challenge — AI Engineer (Artefact)

Project README & Technical Documentation

Author: Flavio Midea | **Stack:** Python · LangChain · MCP · Ollama (`qwen3.5:4b`) · SQLite · Streamlit

1. Project Overview

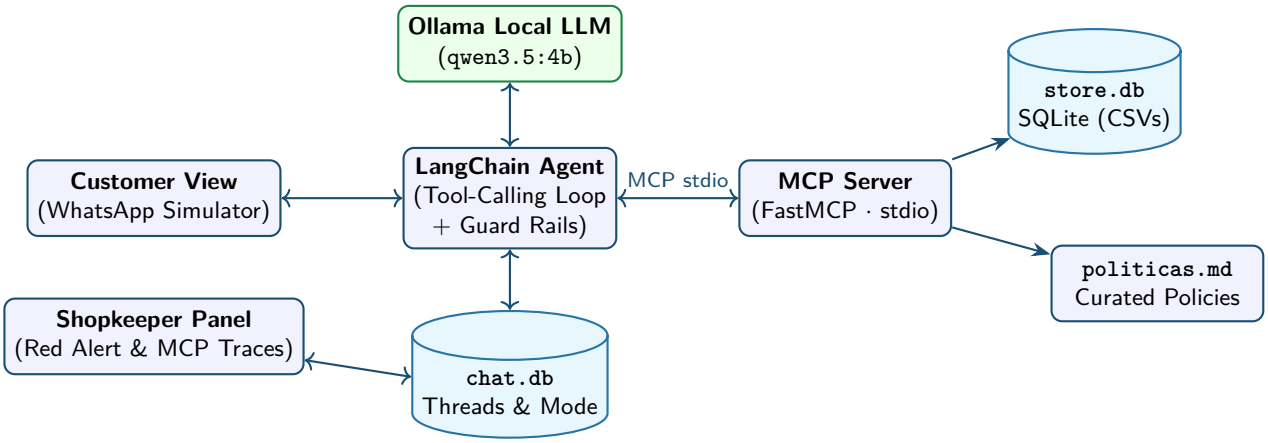
This repository contains a functional prototype of a text-messaging customer-service AI agent built for **Empório da Música**, a fictional musical instrument store located in Campo Grande/MS, Brazil (*"Sua música começa aqui"*), developed for the Artefact AI Engineer Technical Challenge.

The store's support team is overloaded with recurring questions regarding opening hours, order status, product prices, stock availability, promotions, and return/warranty policies. Our agent (**Léo**) runs **100% locally via Ollama**, queries operational data and official policies through a decoupled **Model Context Protocol (MCP) server**, and enforces deterministic business rules in Python/SQLite.

By default, the agent handles customer conversations autonomously. Whenever it faces an out-of-policy request, a formal complaint, an action it is not allowed to execute (such as processing a return or cancelling an order), or missing information, it **escalates the conversation to a human attendant**, triggering a **RED alert** on the shopkeeper's dashboard and remaining silent on that thread until the shopkeeper returns control to the AI.

> **Note (Prototype Scope):** The user interface is built in **Streamlit** to simulate both the customer's WhatsApp view and the shopkeeper's management dashboard in a single self-contained prototype. It is not connected to the live WhatsApp Business API (see Section 6).

2. System Architecture & How It Works



2.1 MCP Tools Exposed (`app/mcp_server/server.py`)

Tool Name	Purpose & When the Agent Calls It
`search_products`	Searches the catalog with exact filters (`query`, `max_price`, `min_price`, `only_available`, `on_promotion`). Used for budget queries (e.g., "guitars up to R$ 1000") or active promotions.
`get_product_details`	Fetches full specs, original/promotional price, PIX price (5% discount, non-cumulative with active promos), max interest-free installments, stock status, and automatic similar alternatives if out-of-stock or discontinued.
`list_categories`	Lists all 9 musical instrument categories sold by the store with active product counts.
`get_customer_orders`	Retrieves orders belonging **exclusively to the current customer**, including status, items, tracking code, cancellation notes, and precomputed return/warranty eligibility (`policy_check`).
`get_store_policies`	Deterministic topic-based retrieval of the store manual (`about_and_contact`, `opening_hours`, `payment`, `returns_exchanges`, `shipping`, `promotions`, `special_situations`, `warranty`, `privacy`).
`get_opening_status`	Checks in real time whether the store is open right now, today's hours, and the next opening time.

In addition, the agent has a local tool `request_human_handoff(reason)` (kept outside the read-only MCP server because it mutates conversation state in `chat.db`).

2.2 Human-in-the-Loop Escalation Lifecycle

Each customer thread in `chat.db` has a `mode` state machine:

1. `agent` (**Green — Default**): The AI agent answers customer messages autonomously.
2. `waiting_human` (**Red Alert**): Triggered when the agent calls `request_human_handoff(reason)` (or via safety guard rails). A prominent red banner appears at the top of the shopkeeper's dashboard showing the customer's name, phone, timestamp, and reason. The agent stops replying on that thread.
3. `human` (**Yellow**): The shopkeeper clicks *"Assumir conversa"* (Take Over) and replies directly to the customer through the chat interface. Clicking *"Devolver ao agente"* restores `agent` mode.

3. Setup & Execution Instructions

3.1 Prerequisites

- **Python 3.10+** (developed and tested on Python 3.12)
- **Ollama** (`https://ollama.com`) installed and running locally
- ~4 GB of free RAM/VRAM for the default `qwen3.5:4b` model

3.2 Step-by-Step Installation

```
# 1. Pull the local model (default: qwen3.5:4b; qwen3.5:9b is also supported via .
    env)
ollama pull qwen3.5:4b

# 2. Create and activate the virtual environment
python -m venv .venv
```

```
.venv\Scripts\activate              # Windows (Linux/macOS: source .venv/bin/activate
    )
pip install -r requirements.txt
copy .env.example .env              # Optional environment overrides

# 3. Launch the Streamlit UI (Customer WhatsApp simulator + Shopkeeper Dashboard)
streamlit run app/ui/streamlit_app.py
```

The UI will be available at http://localhost:8501. The SQLite database `storage_data/store.db` is automatically built from `data/*.csv` on first run (or manually via `python -m app.ingest`).

3.3 CLI, Automated Tests & Scenario Evaluation

Command	Description
`python -m app.cli -customer 1 -fresh`	Interactive terminal chat simulating customer #1.
`python -m pytest`	Runs 16 deterministic unit tests covering data ingestion, accent/plural-insensitive search, PIX vs. promo rules, installment rules, customer order isolation, and return deadlines.
`python -m evals.run_scenarios`	Runs 16 end-to-end scenarios against Ollama, verifying that expected MCP tools are called and measuring processing time.
`python -m evals.run_scenarios -export`	Runs the 16 scenarios and exports rich Markdown reports to `examples/*.md` with exact Python call signatures, JSON arguments, full MCP JSON outputs, and latency metrics.

3.4 Environment Variables (`.env`)

- `OLLAMA_MODEL` (default: `qwen3.5:4b`): Any Ollama model supporting tool calling (e.g., `qwen3.5:9b`).
- `OLLAMA_BASE_URL` (default: `http://localhost:11434`): Local Ollama endpoint.
- `FAKE_NOW` (optional, e.g., `2026-03-20T10:00:00`): Overrides the system clock to deterministically test return windows and opening hours.

4. Technical Decisions & Justifications

Topic	Decision	Justification
Agent Framework	**LangChain** (`create_agent`, `langchain-ollama`, `langchain-mcp-adapters`)	Mature, industry-standard, and 100% open-source/free. Provides native tool-calling loops, Ollama integration, and MCP tool adapters/interceptors, with a seamless upgrade path to LangGraph if stateful workflows are needed later.

Topic	Decision	Justification
Single Agent vs. Multi-Agent (A2A)	**Single Agent + MCP** (A2A documented as evolution)	The store's scope fits cleanly into 7 well-typed tools. With a small local LLM (4B–9B), every inter-agent hop adds latency and compounding routing errors. **Agent-to-Agent (A2A)** is recommended as a future evolution once separate domain agents exist (e.g., Sales, Logistics/Post-Sale, Billing).
Tooling Layer	**MCP Server** (Official SDK / FastMCP over `stdio`)	Decouples data access and business rules from the LLM orchestrator. The exact same MCP server can be reused by a production WhatsApp webhook, Claude Desktop, or another framework. The agent keeps a persistent MCP session in a background event loop to avoid subprocess startup overhead on each message.
LLM & Provider	**Ollama Local** (`qwen3.5:4b` default, `qwen3.5:9b` configurable)	Zero API cost, full data privacy (LGPD—customer data never leaves the store's machine), and runs on modest hardware. Configured with `reasoning=False` (thinking disabled) and `temperature=0.2` to cut latency while maintaining reliable tool routing.
Policy Retrieval (RAG vs. Topic Tool)	**Structured Topic Lookup** (No Vector DB)	The store manual (`políticas_da_loja.pdf`, 8 pages, ∼4k tokens) is small and cleanly divided into 10 sections. Curating it into `politicas.md` by section and exposing `get_store_policies(topic)` with an explicit `Literal` enum yields 100% deterministic, complete, and auditable retrieval without risking chunk truncation or embedding mismatches.
Operational Data Retrieval	**CSV → SQLite** + Parameterized Queries	LLMs should never do arithmetic or filter raw CSVs in context. SQLite handles price ceilings, active promotion joins, stock checks, and accent/plural-insensitive search (`violões` = `violao`) deterministically.
Business Rules in Code	Deterministic Python calculation in `repository.py`	Promotional prices, the 5% PIX discount (**non-cumulative** with active promotions per manual §6.2), maximum interest-free installments (§3.1), and 7/30/90-day return/warranty windows (§4, §8) are pre-computed in Python and returned ready-to-read to the model.
Prompt Engineering	Persona **Léo** (`prompts.py`) + Strict PT-BR rule + Few-shot mapping	Tool names and schemas are in English (models follow English schemas more reliably), while Rule #1 forces the agent to **always reply in Brazilian Portuguese**. Includes dynamic context (customer name, ID, current weekday/time) and explicit rules for out-of-scope, gibberish inputs, and accessories (which the store does not sell).

Topic	Decision	Justification
Security & LGPD Privacy	**MCP Tool Interceptor** (`_force_customer_id`)	The `customer_id` passed to `get_customer_orders` is overwritten at the interceptor level with the authenticated session's `customer_id`. Even under prompt injection, a customer can never read another customer's orders.
Safety Guard Rails	3-layer runtime guard rails in `agent.py`	(1) **Stall/Policy Retry:** If the model says "I will check" without calling a tool, or answers a follow-up policy question from memory, the runtime automatically re-prompts it to invoke the required tool. (2) **Auto-Handoff:** If the model's text promises that it notified the team without calling `request_human_handoff`, the runtime triggers the handoff automatically. (3) **Exception Fallback:** Any runtime error triggers a graceful apology and human handoff.

4.1 Performance Benchmark: Why `qwen3.5:4b` Is Default

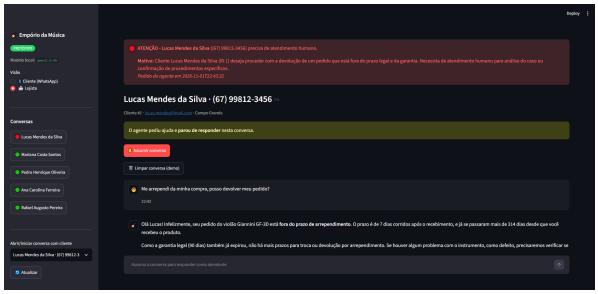
Measured on the development machine (NVIDIA GTX 1650 4 GB VRAM, Intel Core i7-4790, 16 GB RAM):

- `qwen3.5:9b` (~6.4 GB Q4_K_M): Partially offloads to CPU RAM (~4.3 tokens/s generation), taking ~**210s (3.5 min)** for a single tool-calling turn—unusable for interactive chat.
- `qwen3.5:4b` (~3.3 GB Q4_K_M): Fits much better in the 4 GB GPU (~8.2 tokens/s generation, ~51 tokens/s prompt eval), answering tool-calling turns in ~**25–60s**.

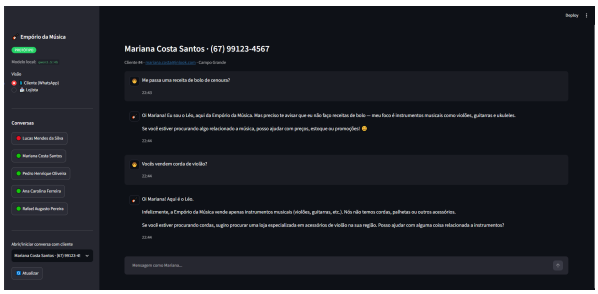
Therefore, `qwen3.5:4b` is set as the default, while `qwen3.5:9b` remains configurable via `OLLAMA_MODEL=qwen3.5:9` for machines with ≥8 GB VRAM.

5. User Interface & Interaction Examples

5.1 Streamlit Prototype Screenshots



(a) Shopkeeper View: Red alert banner & MCP trace



(b) Customer View: Scope refusal & redirection

5.2 Summary of the 16 Evaluated & Exported Scenarios (`examples/*.md`)

All 16 scenarios were executed against the live agent (`python -m evals.run_scenarios -export`) and passed (**16/16 PASS**). Each exported Markdown file records the **processing time per turn**,

the **exact Python tool call signature**, the **JSON arguments sent**, and the **full JSON output returned by the MCP server**.

#	File	Scenario	MCP Tools Verified & Highlights	Tim
01	`01_catalogo.md`	Guitars up to R$ 1000 + Yamaha C40 details	`search_products(max_price=1000)` → `get_product_details('Yamaha C40')`	122.
02	`02_loja.md`	Store address + "Are you open right now?"	`get_store_policies('about_and_contact')` → `get_opening_status()`	72.
03	`03_preco_promocao.md`	Takamine GD20 price + active promotions	`get_product_details('Takamine GD20')` → `search_products(on_promotion=True)`	53.
04	`04_devolucao.md`	**Non-trivial:** Regret return on delivered order	`get_customer_orders(customer_id=1)` (315 days > 7-day limit) → `request_human_handoff`	95.
05	`05_fora_escopo.md`	Carrot cake recipe + guitar strings request	Direct polite refusal for out-of-scope & accessory redirection (no tools called)	58.
06	`06_produto_esgotado.md`	Out-of-stock Giannini GF-3D	`get_product_details('Giannini GF-3D')` → reports stock 0 + suggests 3 available alternatives	48.
07	`07_produto_descontinuado.md`	Discontinued 7-string Shelby SN-7C	`get_product_details('Shelby SN-7C')` → informs discontinued status + suggests alternatives	51.
08	`08_pix_nao_cumulativo.md`	Crafter HT-100 promo + PIX discount question	`get_product_details('Crafter HT-100')` → shows 18% promo & explains 5% PIX does not stack	51.
09	`09_rastreio_pedido.md`	Order #8 status & tracking code	`get_customer_orders(customer_id=2, order_id='8')` → `shipped`, code `BRJL5544332BR`	65.
10	`10_pedido_cancelado.md`	Why was my order cancelled?	`get_customer_orders(customer_id=17)` → reads cancellation `notes` (item out of stock)	46.
11	`11_parcelamento...sp.md`	**Multi-turn:** Credit card installments + shipping to São Paulo	Turn 1: `get_store_policies('payment')` (up to 12x rules) → Turn 2: `get_store_policies('shipping')` (Correios/Jadlog deadlines)	172.
12	`12_frete_entrega.md`	Shipping to Campo Grande vs. other cities	`get_store_policies('shipping')` → motoboy (free > R$ 500) vs. PAC, SEDEX, Jadlog	54.
13	`13_garantia.md`	Warranty rules + broken string question	`get_store_policies('warranty')` → 90-day legal + manufacturer warranty; broken strings excluded as natural wear	52.
14	`14_categorias.md`	Product categories sold by the store	`list_categories()` → lists active instrument categories and counts	49.
15	`15_reclamacao_handoff.md`	Warped guitar neck complaint	Empathy + escalates immediately via `request_human_handoff` (red alert)	32.

#	File	Scenario	MCP Tools Verified & Highlights	Tim
16	`16_entrada_aleatoria.md`	**Gibberish input** (`askdjansdkj`) + recovery	Turn 1: asks for clarification without calling tools → Turn 2: `search_products('violao nylon', max_price=700)`	110.

5.3 Sample Rich Trace Excerpt (from `examples/04_devolucao.md` & `11_parcelamento_e_entreg`

Below is an excerpt illustrating how the exported `.md` files document both the agent's exact function call and the MCP server's response:

```
### Turno 1 (Scenario 04 - Return Request)
Cliente (Lucas): Me arrependi da minha compra, posso devolver meu pedido?
Leo (55.55s): Ola Lucas! Infelizmente, seu pedido do violao Giannini GF-3D ja esta
    *fora do prazo de arrependimento* (7 dias corridos apos o recebimento)... Quer
    que eu encaminhe para um atendente humano?

1. Chamada realizada pelo agente:
   get_customer_orders(customer_id=1)
Resposta obtida da funcao (MCP / Tool Output):
   { "found": true, "today": "2026-11-02",
     "orders": [{ "order_id": 4, "status": "delivered", "estimated_delivery":
         "2025-12-22",
       "policy_check": { "days_since_receipt_approx": 315,
         "regret_return_7_days": "FORA do prazo (7 dias corridos apos o recebimento
             )",
         "legal_warranty_90_days": "expirada" } }] }

### Turno 2 (Scenario 04 - Handoff Confirmation)
Cliente (Lucas): Sim, quero devolver.
1. Chamada realizada pelo agente:
   request_human_handoff(reason='Cliente Lucas Mendes da Silva (customer_id=1)
       deseja prosseguir com a devolucao de um pedido, mas o prazo legal de
       arrependimento ja expirou...')
-> RED ALERT triggered on Shopkeeper Dashboard (mode = waiting_human)
```

6. Known Limitations & Future Improvements

Known Limitations

- **Simulated WhatsApp UI:** Streamlit is used for rapid prototyping; there is no live webhook to the WhatsApp Business Cloud API yet, and the shopkeeper dashboard requires a manual refresh.
- **Local Hardware Latency:** Running a 4B/9B model on a 4 GB GTX 1650 yields 25–60s latency per turn without token streaming.
- **Small Model Quirks:** A 4B model can occasionally omit items from a list or skip a tool call in multi-turn conversations. This is mitigated via deterministic Python calculations and runtime retry guard rails, but not 100% eliminated.
- **Receipt Date Proxy:** `orders.csv` only provides `estimated_delivery`, which is used as a proxy for the delivery date when computing the 7-day regret window (flagged in `policy_check.obs`).
- **Phone vs. WhatsApp Distinction:** The manual lists two numbers; as clarified in the business rules, `(67) 3341-4444` is for voice calls only and `(67) 3321-4500` is exclusively for WhatsApp messages.

What Would Be Done With More Time

- **FastAPI + WebSocket Frontend & WhatsApp Webhook:** Replace Streamlit with a FastAPI backend pushing real-time WebSocket alerts to multiple authenticated shopkeepers and connecting directly to the WhatsApp Business API.
- **Multi-Agent Architecture (A2A):** Evolve into specialized agents communicating via A2A (e.g., Catalog/Sales Advisor, Post-Sale & Logistics Agent, Billing Agent) once new external integrations are added.
- **LangGraph Stateful Flows:** Use LangGraph for guided multi-step workflows (e.g., collecting defect photos and serial numbers before human handoff) and human-approved write tools.
- **Production Inference & Observability:** Deploy on a dedicated GPU / vLLM or hosted API for < 3s latency with token streaming, semantic caching for FAQs, and Langfuse/LangSmith tracing + LLM-as-a-judge evaluations.

7. Use of AI Coding Assistants & Repository Structure

Workflow with AI Coding Assistant: This project was developed using **Antigravity** (Google DeepMind's agentic coding assistant) in a pair-programming workflow:

1. **Problem Framing & Architecture (Human):** I defined the core architecture: local Ollama model for zero cost/privacy, MCP server for data/policy tools, A2A reserved for multi-agent evolution, WhatsApp-like UI populated from `customers.csv`, and autonomous operation with a red alert human handoff for the shopkeeper.
2. **Data Exploration & Planning:** The assistant inspected all CSVs and the policy PDF, surfacing key edge cases (only 4 of 25 promotions active, non-cumulative PIX discount, no accessories sold, all delivered orders past the 7-day regret window) in an implementation plan.
3. **Technical Decisions (Human):** I reviewed the plan and chose LangChain, Streamlit as an explicit prototype (with FastAPI documented as future work), English tool schemas with strict Portuguese responses, and versioning `data/`.
4. **Incremental Implementation & Empirical Tuning:** Built in small commits with deterministic `pytest` tests first. When benchmarking showed `qwen3.5:9b` took ~3.5 min/turn on my GPU, I decided to switch the default to `qwen3.5:4b` (~2x faster) and added runtime guard rails after observing multi-turn tool-calling edge cases.

Repository Structure:

```
app/
  agent/              agent.py (LangChain + MCP + Ollama + guard rails), prompts.py
  mcp_server/         server.py (FastMCP tools over stdio)
  knowledge/          politicas.md (curated store manual by topic)
  storage/            chat_store.py (SQLite threads, messages, tool traces & mode)
  ui/                 streamlit_app.py (WhatsApp customer view + Shopkeeper alert
      panel)
  repository.py       SQLite queries + deterministic business rules (promo/PIX/
      returns)
  ingest.py           CSV -> SQLite loader
  config.py, cli.py Configuration & terminal chat CLI
data/                 Store CSVs and policies PDF provided by the challenge
tests/                16 deterministic pytest unit tests (no LLM required)
evals/                run_scenarios.py (16 end-to-end evals + rich Markdown exporter)
examples/             16 exported real conversations with full MCP JSON traces &
    timing
imgs/                 UI screenshots (Shopkeeper red alert & Customer chat)
```